

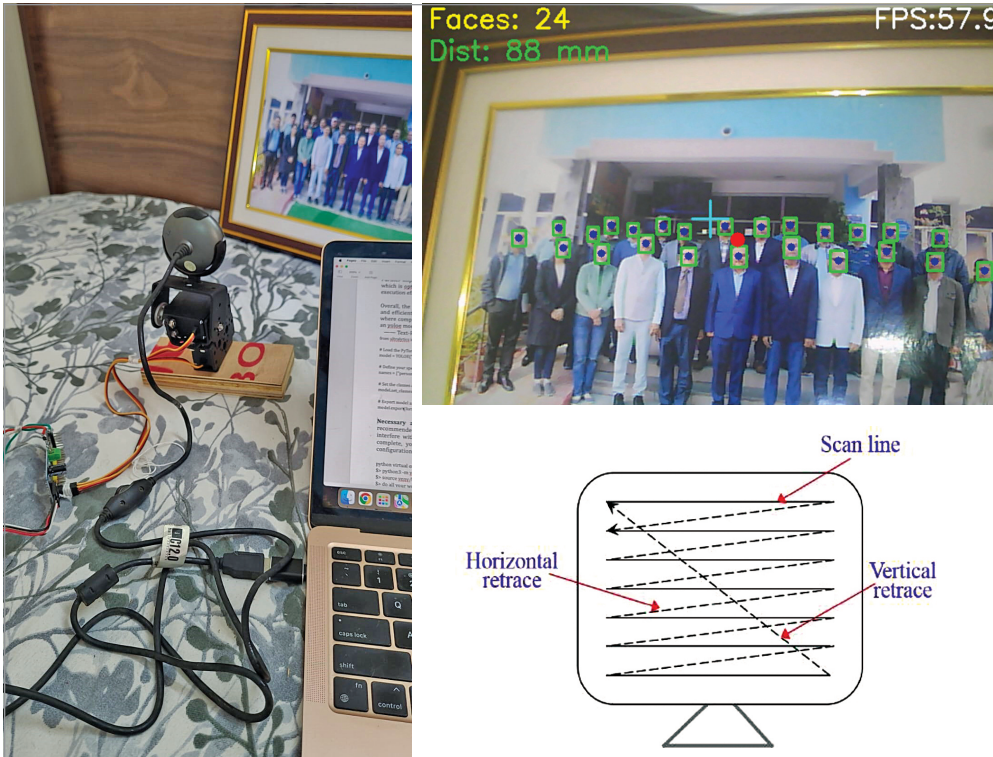


Fig. 5: Real-time face tracking with a frame rate exceeding 50 FPS

prompt-based detection. Objects can be specified using text prompts for classes supported by the base model or image prompts for new and custom objects. The model encodes these prompts and matches them with its learned visual representations, enabling the detection of visually similar objects during inference without additional training. This capability supports highly specific, context-aware detection, such as identifying a ‘person wearing glasses’ rather than simply a ‘person,’ detecting a smartphone, or recognising a unique personal item from only a few reference images. Restricting detection to user-defined prompts can also reduce the computational workload and improve inference speed.

To maximise performance, the YOLOE model is converted to the NCNN format, which is optimised for CPU-based inference and helps reduce runtime overhead. In the present system, the laptop performs all YOLOE (or YOLOv8-face)

inference, achieving substantially higher frame rates than embedded platforms such as the Raspberry Pi. The ESP32 is dedicated to real-time pan-tilt servo control and sensor interfacing. Communication between the laptop and ESP32 uses the lightweight ESPConnect protocol, enabling low-latency and reliable command transmission.

Two practical customisation routes are available. When the desired objects are already supported by the pre-trained YOLOE model (for example, yoloe-11s-seg.pt), they can be selected using text prompts. These prompts are embedded directly into the exported model, resulting in a lighter and more focused NCNN model. The conversion is performed using a short Python script, as shown in Fig. 4.

This approach typically achieves 15-30 frames per second (FPS) when detecting multiple objects on a laptop and can reach approximately 50 FPS when specialised for a single class, such as person tracking. For

objects not supported by the base model, YOLOE provides image prompting. A small number of reference images captured from different viewpoints are supplied to the model, which extracts their visual characteristics and associates them with a unique internal representation. This enables the model to detect the target object during subsequent inference without additional training.

At present, text prompting and image prompting cannot be combined within a single YOLOE model pipeline; they must be implemented as separate models. Future versions of YOLOE may support

hybrid prompting.

Note. All Python work is performed within a virtual environment to avoid interfering with the system-wide Python installation as follows:

```
python3 -m venv venv
source venv/bin/activate
# perform all development work
deactivate
```

The minimum required packages are installed with:

```
pip install ultralytics opencv-python-headless numpy onnx onnxruntime torch torchvision torchaudio
```

Because the standard YOLOE models do not include a predefined ‘face’ class, a separate pre-trained YOLOv8-face model is obtained from <https://github.com/lindevs/yolov8-face/releases>

The model-size suffixes indicate nano (n), small (s), medium (m), and large (l) variants; smaller models provide faster inference, while larger models generally offer higher accuracy. After downloading the model, its PyTorch weights are converted to NCNN format:

```
yolo export
```